

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I Annajothi S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Annajothi S

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and	Organized and understandable	Limited clarity and structure	Poor clarity; difficult to
		easy to understand	presentation		understand

16.a) Online Food Delivery System

A startup wants to develop an online food delivery platform where users can browse restaurants, place orders, and track delivery in real time. The system should also allow restaurants to update menus and delivery agents to manage orders.

Question: Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

Introduction

An **Online Food Delivery System** enables customers to browse restaurants, view menus, place food orders, make payments, and track deliveries in real time. Restaurants can update their menus and manage incoming orders, while delivery agents receive assignments and update delivery status. The system involves multiple users and requires scalability, security, and continuous feature enhancements.

1. Application of Object-Oriented Principles

A. Encapsulation

Definition:

Encapsulation is the process of bundling data and methods together in a class while restricting direct access to internal data using private attributes and public methods.

Application in the Food Delivery System

For example, the Customer class stores personal information privately and allows access only through methods.

```
class Customer {  
    private String customer Name;  
    private String address;  
    private String phone Number;  
  
    public void place Order() { }  
    public void update Profile() { }  
}
```

Benefits

- Protects sensitive information such as phone numbers and addresses.
- Prevents unauthorized modification of customer and payment details.
- Improves data security and maintainability.

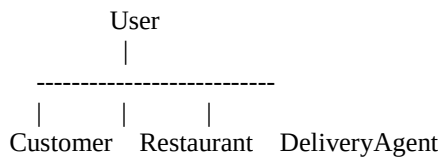
B. Inheritance

Definition:

Inheritance allows one class to acquire the properties and behaviors of another class, reducing code duplication.

Application in the Food Delivery System

A common User class can be extended by different user types.



- **User**
 - userId
 - name
 - login()
- **Customer**
 - Place Order()
- **Restaurant**
 - Update Menu()
- **Delivery Agent**
 - Accept Delivery()
 - Update Delivery Status()

Benefits

- Reuses common attributes and methods.
- Simplifies code maintenance.
- Makes the system easier to extend with new user roles.

C. Polymorphism

Definition:

Polymorphism allows the same method name to perform different functions depending on the object.

Example

```
class User {
    void login() { }
}

class Customer extends User {
    void login() {
        System.out.println("Customer Login");
    }
}

class Restaurant extends User {
    void login() {
        System.out.println("Restaurant Login");
    }
}

class Delivery Agent extends User {
    void login() {
        System.out.println("Delivery Agent Login");
    }
}
```

```
}  
}
```

Here, the `login()` method behaves differently for each user type.

Benefits

- Increases flexibility.
- Supports dynamic behavior.
- Makes adding new user roles easier without changing existing code.

2. Other Important Classes in the System

Class	Responsibility
Customer	Browse restaurants, place orders, make payments
Restaurant	Manage menu items and process orders
Order	Store order details and status
DeliveryAgent	Accept deliveries and update delivery progress
Payment	Handle online payment transactions
Menu	Store available food items and prices

3. Suitable SDLC Model

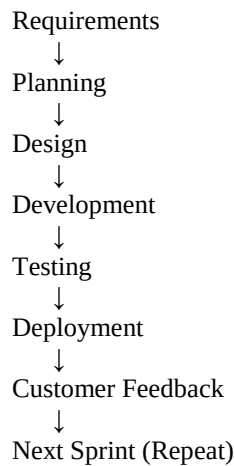
Agile SDLC Model

The **Agile Model** is the most suitable choice for an online food delivery platform.

Why Agile?

1. **Frequent Requirement Changes**
 - New features such as coupons, live tracking, loyalty programs, and payment options can be added incrementally.
2. **Incremental Development**
 - The startup can release a basic version with registration and ordering, then add restaurant management, tracking, and notifications in later iterations.
3. **Continuous Customer Feedback**
 - User reviews and feedback help improve usability and introduce new functionality quickly.
4. **Faster Delivery**
 - Working software is delivered in short development cycles (sprints), enabling rapid deployment.
5. **Easy Maintenance**
 - Bugs and enhancements can be addressed in subsequent iterations without waiting for a complete redevelopment cycle.

4. Agile Development Cycle



5. Justification for Choosing Agile

- Supports changing business requirements and market trends.
- Delivers working features quickly through iterative releases.
- Encourages collaboration among developers, restaurants, delivery partners, and customers.
- Facilitates continuous testing and improvement.
- Reduces project risk by validating features early and often.

6. Conclusion

The **Online Food Delivery System** effectively demonstrates object-oriented programming concepts:

- **Encapsulation** protects sensitive customer, restaurant, and order data.
- **Inheritance** enables different user types (Customer, Restaurant, Delivery Agent) to share common functionality while extending specialized behavior.
- **Polymorphism** allows common operations, such as `login()` or `updateStatus()`, to behave differently based on the user type.

Among SDLC approaches, the **Agile Model** is the best fit because it supports iterative development, rapid feature delivery, continuous customer feedback, and easy adaptation to changing business requirements, making it ideal for a startup building a modern food delivery platform.

b) Hospital Management System

A hospital plans to digitize its operations including patient registration, doctor scheduling, billing, and medical records management. Different users such as doctors, nurses, admin staff, and patients interact with the system.

Question: Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

Introduction

A **Hospital Management System (HMS)** is designed to automate hospital operations such as patient registration, doctor scheduling, appointment management, billing, and medical record maintenance. The system is used by **patients, doctors, nurses, and administrative staff** to improve efficiency, reduce paperwork, and provide better healthcare services.

1. Object-Oriented Design Approach

Object-oriented design models the system as a collection of interacting objects. Each object represents a real-world entity with its own attributes and methods.

A. Encapsulation

Definition: Encapsulation combines data and methods into a single class while protecting internal data from unauthorized access.

Example

```
class Patient {  
    private int patientId;  
    private String name;  
    private String medicalHistory;  
  
    public void registerPatient() { }  
    public void viewMedicalRecord() { }  
}
```

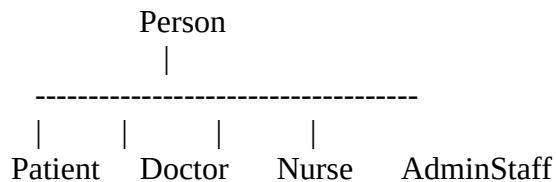
Benefits

- Protects confidential patient information.
- Ensures secure access to medical records.
- Improves data integrity and maintainability.

B. Inheritance

Definition: Inheritance allows specialized classes to inherit common properties from a base class.

Example Structure



- **Person**
 - personId
 - name
 - age
 - contactNumber
- **Patient**
 - register()
 - viewRecords()
- **Doctor**
 - diagnosePatient()
 - prescribeMedicine()
- **Nurse**
 - updateVitals()
 - assistDoctor()
- **AdminStaff**
 - scheduleAppointment()
 - generateBill()

Benefits

- Reduces duplicate code.
- Encourages code reuse.
- Simplifies future enhancements.

C. Polymorphism

Definition: Polymorphism allows the same method to perform different actions depending on the object invoking it.

Example

```
class Person {
    void login() { }
}

class Doctor extends Person {
    void login() {
        System.out.println("Doctor Login");
    }
}

class Patient extends Person {
    void login() {
        System.out.println("Patient Login");
    }
}
```



```

    }
}

class AdminStaff extends Person {
    void login() {
        System.out.println("Admin Login");
    }
}

```

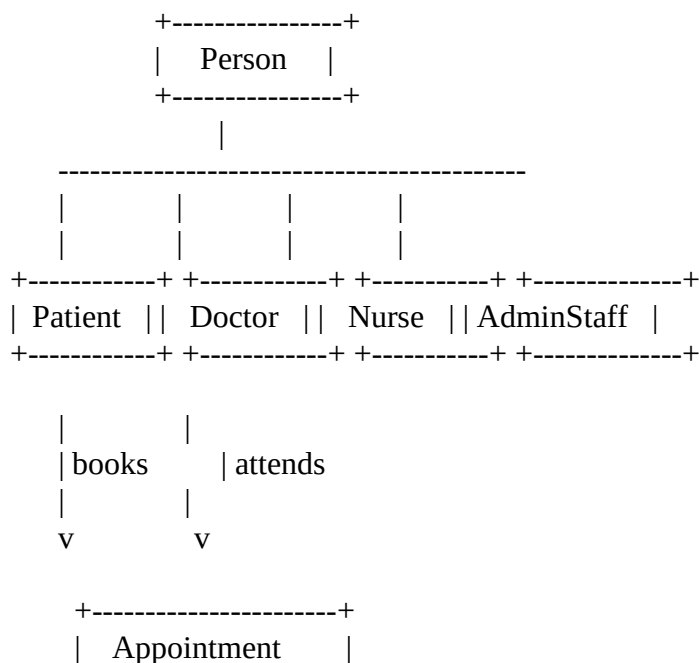
Benefits

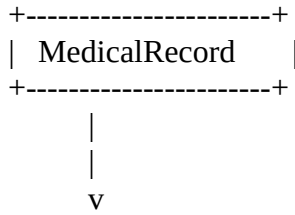
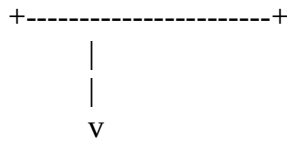
- Supports flexible implementation.
- Makes the system scalable.
- Simplifies handling of multiple user roles.

2. Key Classes in the Hospital Management System

Class	Responsibilities
Patient	Register, book appointments, view medical records
Doctor	Examine patients, diagnose illnesses, prescribe treatments
Nurse	Record patient vitals and assist doctors
AdminStaff	Manage registrations, appointments, and billing
Appointment	Store appointment details and schedules
MedicalRecord	Maintain diagnosis, prescriptions, and treatment history
Bill	Generate invoices and manage payments

3. Relationships Between Classes





Relationship Explanation

- A **Patient** books one or more **Appointments**.
- A **Doctor** attends appointments and updates **Medical Records**.
- Each **Medical Record** is linked to a **Patient** and may generate a **Bill**.
- **Admin Staff** manages appointments and billing processes.
- **Nurses** assist doctors and update patient information.

4. Recommended SDLC Model

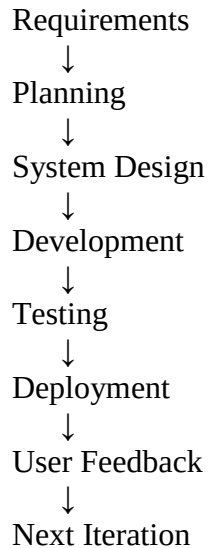
Agile SDLC Model

The **Agile Model** is the most suitable choice for developing a Hospital Management System.

Reasons

1. **Changing Requirements**
 - Hospitals may require new modules such as telemedicine, pharmacy management, laboratory integration, or insurance processing.
2. **Incremental Development**
 - The system can first deliver patient registration and appointment scheduling, then add billing, medical records, and reporting modules.
3. **Continuous Feedback**
 - Doctors, nurses, administrators, and patients can provide feedback that is incorporated into future iterations.
4. **Improved Quality**
 - Frequent testing and reviews help detect and fix issues early.
5. **Faster Delivery**
 - Functional components can be released quickly instead of waiting for the entire system to be completed.

5. Agile Development Cycle



6. Advantages of Using Agile for HMS

- Supports evolving healthcare requirements.
- Delivers usable features in short development cycles.
- Encourages collaboration between developers and hospital staff.
- Enables continuous improvement based on user feedback.
- Reduces project risk through regular testing and incremental releases.

7. Conclusion

The Hospital Management System can be effectively modeled using object-oriented principles:

- Encapsulation protects sensitive information such as patient records and billing data.
- Inheritance allows common attributes to be shared among users like doctors, nurses, patients, and administrators.
- Polymorphism enables common operations (such as login or notifications) to behave differently depending on the user type.

The Agile SDLC Model is the most appropriate choice because it supports iterative development, accommodates changing healthcare requirements, enables continuous stakeholder feedback, and ensures timely delivery of reliable and scalable hospital management features.